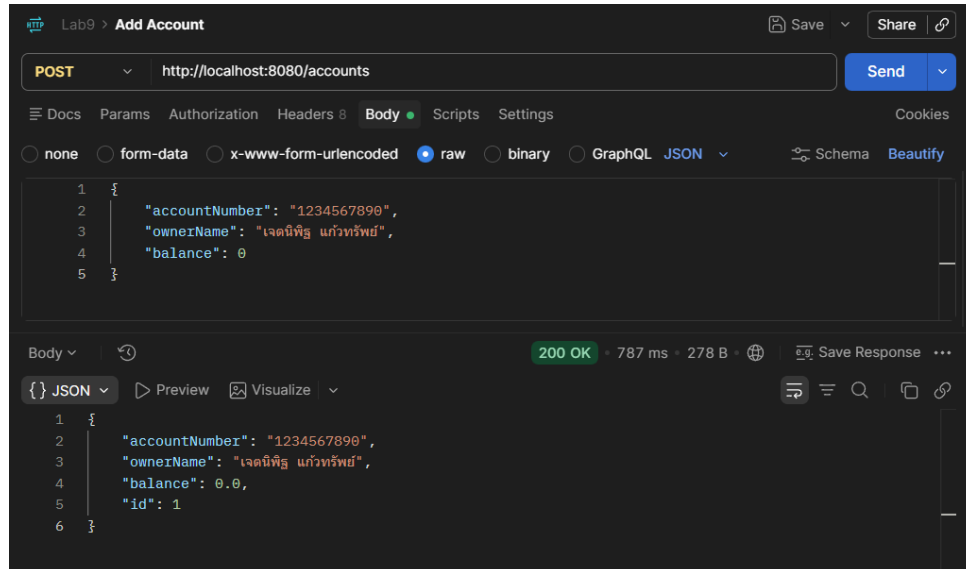


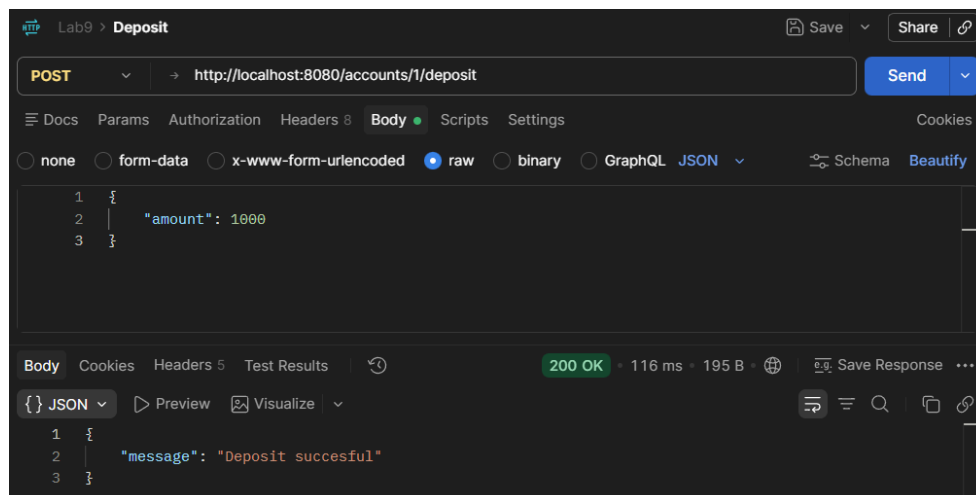
รายงาน Lab 9

1. การสร้าง Account



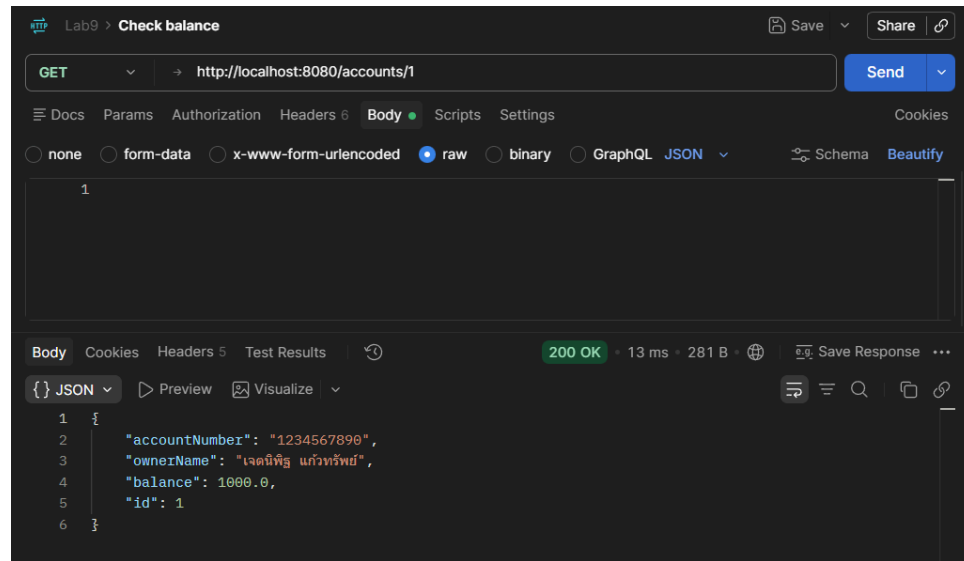
จากภาพแสดงผลการทดสอบการสร้าง Account ด้วย Postman พบว่าสามารถส่งค่าขอสร้าง Account ไปยังระบบได้สำเร็จ โดยระบบตอบกลับด้วยสถานะ 200 OK แสดงว่าการดำเนินการเสร็จสมบูรณ์ และข้อมูล Account ถูกส่งไปบันทึกใน Database ตามที่กำหนด

2. การฝากเงินสำเร็จ



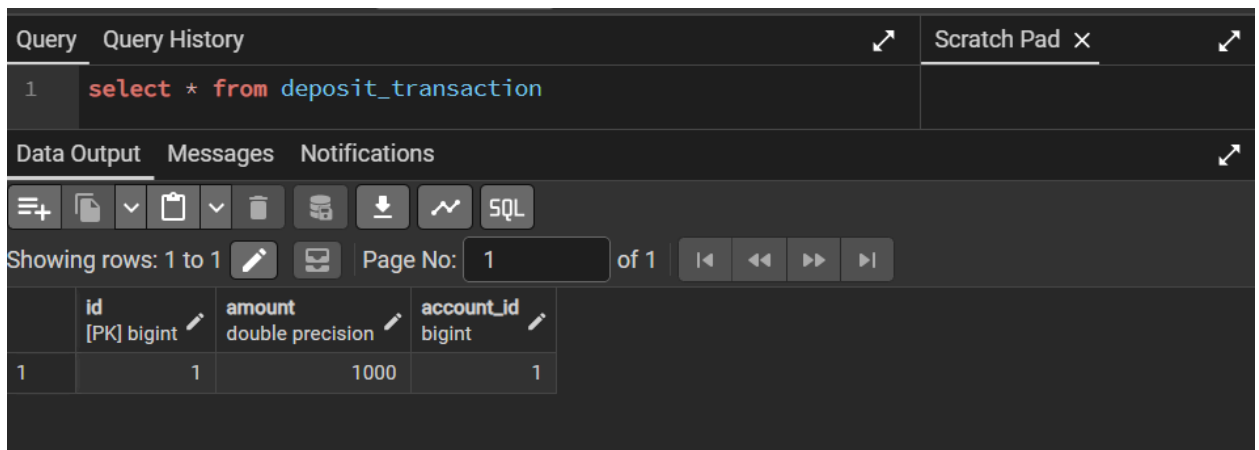
จากภาพแสดงผลการทดสอบการฝากเงินผ่าน Postman พบว่าสามารถฝากเงินเข้าบัญชีได้สำเร็จ โดยระบบตอบกลับด้วยสถานะ 200 OK และแสดงข้อความ "Deposit successful" แสดงว่าระบบสามารถดำเนินการฝากเงินและบันทึกข้อมูลที่เกี่ยวข้องได้สำเร็จ

3. ตรวจสอบยอดเงินใน Account



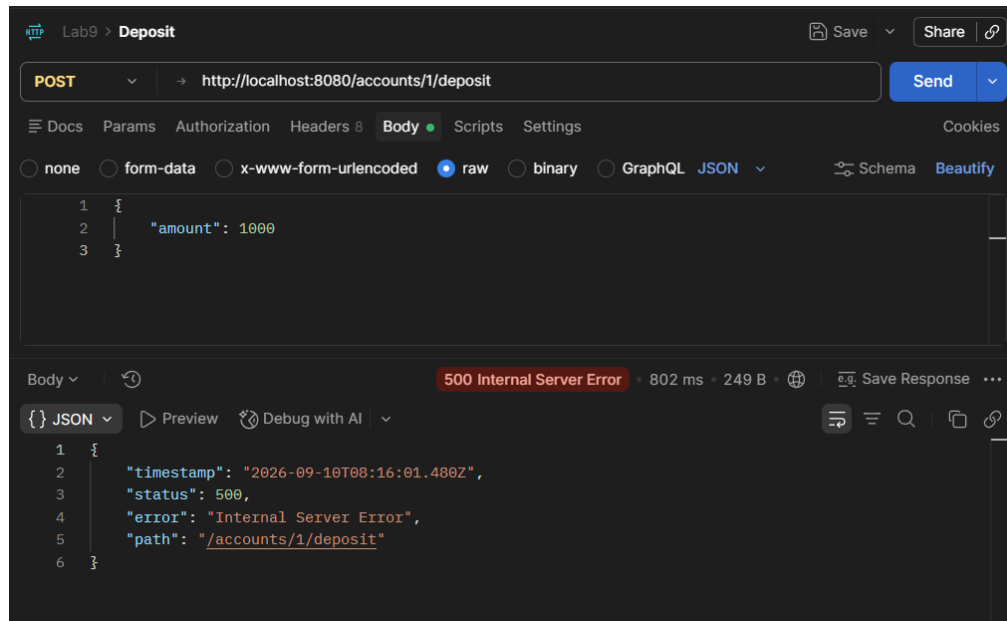
จากภาพแสดงผลการตรวจสอบข้อมูล Account หลังจากทำการฝากเงิน พบว่ายอดเงินคงเหลือ (balance) เท่ากับ 1000 ซึ่งเพิ่มขึ้นตามจำนวนเงินที่ฝากในข้อ 2 แสดงว่าระบบสามารถปรับปรุงและบันทึกยอดเงินใน Account ได้ถูกต้อง

4. ตรวจสอบ DepositTransaction



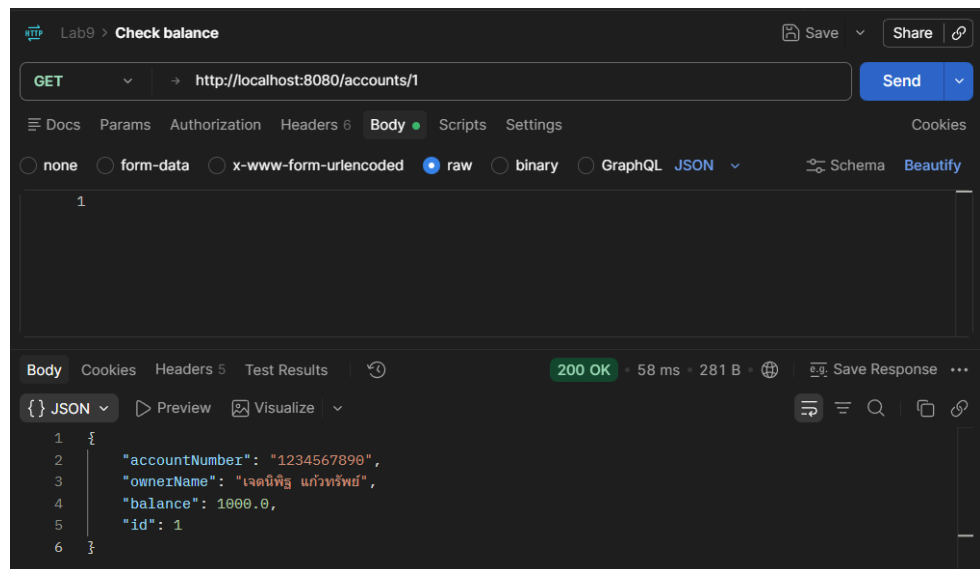
จากภาพแสดงผลการตรวจสอบข้อมูล DepositTransaction ใน Database พบว่ามีการบันทึกรายการฝากเงินสำเร็จ โดยมี id เท่ากับ 1, จำนวนเงินฝาก (amount) เท่ากับ 1,000 บาท และ account_id เท่ากับ 1 ซึ่งตรงกับ Account ที่ทำการฝากเงิน แสดงว่าระบบสามารถบันทึกข้อมูลธุรกรรมการฝากเงินลงใน Database ได้อย่างถูกต้อง

5. ทดลอง Transaction Rollback



จากภาพแสดงผลการทดลอง Rollback พบว่าเมื่อเกิดข้อผิดพลาดระหว่างการฝากเงิน ระบบตอบกลับด้วยสถานะ 500 Internal Server Error ซึ่งแสดงว่าการทำรายการฝากเงินไม่สำเร็จ

5.1 ตรวจสอบยอดเงินในบัญชีอีกครั้ง



จากภาพแสดงผลการตรวจสอบข้อมูล Account อีกครั้ง หลังจากทำการฝากเงิน โดยกำหนดให้เกิด error หลังดำเนินการ พบว่ายอดเงินคงเหลือ (balance) เท่ากับ 1000 ซึ่งไม่ถูกเปลี่ยนแปลง แสดงว่าระบบสามารถ rollback กรณีที่ดำเนินการไม่สำเร็จได้

5.2 ตรวจสอบตาราง deposit_transaction

Query

Query History

Scratch Pad

1

```
select * from deposit_transaction
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

Page No: 1 of 1

	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

จากภาพแสดงผลการตรวจสอบข้อมูล DepositTransaction ใน Database พบว่าไม่มีการบันทึกรายการฝากเงินสำเร็จ แสดงว่าระบบสามารถ Rollback ในกรณีที่ดำเนินการไม่สำเร็จได้

จากการทดลอง Transaction Rollback นี้สามารถสรุปได้ว่า การใช้ @Transactional ระบบจะยกเลิกการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นภายใน Transaction ทำให้ยอดเงินใน Account ไม่ถูกเปลี่ยนแปลง และไม่มีการบันทึก DepositTransaction รายการใหม่ลงใน Database

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

จากการทดลองในข้อ 5 เมื่อกำหนด @Transactional ให้กับเมธอด deposit() แม้ว่าระบบจะบันทึก Account และ DepositTransaction สำเร็จแล้ว แต่เมื่อเกิด RuntimeException("Test Rollback") ระบบจะทำการ Rollback การเปลี่ยนแปลงทั้งหมดภายใน Transaction ส่งผลให้ยอดเงินใน Account กลับเป็นค่าเดิมและรายการ DepositTransaction ที่สร้างขึ้นถูกยกเลิก

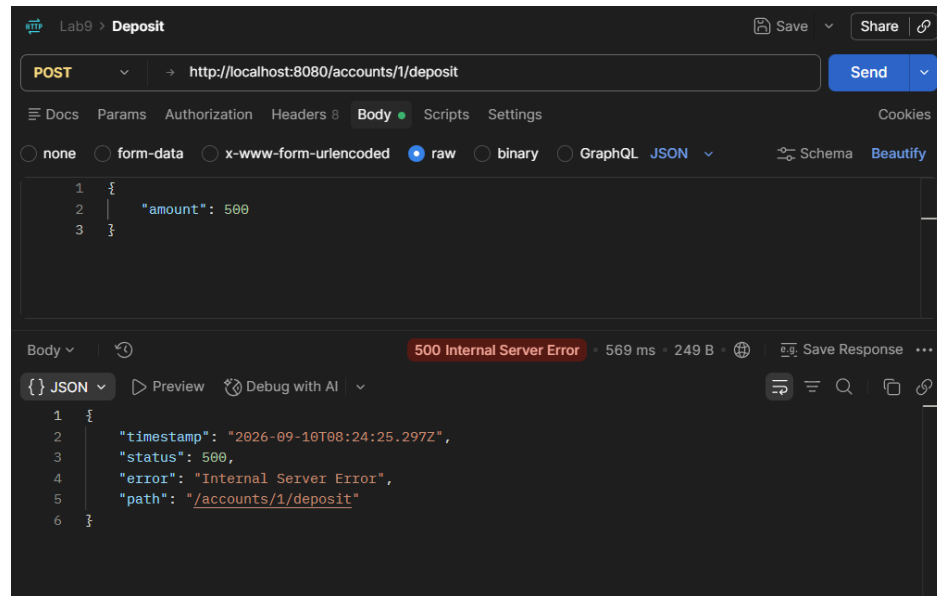
ในทางกลับกัน เมื่อลบ @Transaction ดังภาพ

```
// @Transactional
public void deposit(Long accountId, double amount) {
    // find account
    Account account = accountRepository.findById(accountId)
        .orElseThrow(() -> new RuntimeException("Account not found " + accountId));

    // update balance in account
    double new_balance = account.getBalance() + amount;
    account.setBalance(new_balance);
    accountRepository.save(account);

    // save transaction
    DepositTransaction deposit = new DepositTransaction(amount, account);
    depositRepository.save(deposit);
    throw new RuntimeException("Test Rollback");
}
```

และทดสอบการฝากเงินผ่าน Postman ดังภาพ



จากภาพจะเห็นว่าระบบตอบกลับด้วยสถานะ 500 Internal Server Error ซึ่งแสดงว่าการทำรายการฝากเงินไม่สำเร็จ

จากนั้นตรวจสอบตาราง `deposit_transaction`

Query

Query History

Scratch Pad

1

`select * from deposit_transaction`

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 2 of 1

	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1
2	4	500	1

จากภาพการตรวจสอบข้อมูล `DepositTransaction` ใน Database พบว่ามีการบันทึกรายการฝากเงินสำเร็จ เพิ่มขึ้นมาหนึ่งรายการ ขณะที่การทดสอบฝากเงินก่อนหน้านี้ได้ผลลัพธ์ว่าฝากเงินไม่สำเร็จ แสดงว่าระบบมีการบันทึกข้อมูลธุรกรรมการฝากเงินลงใน Database โดยไม่มีการ Rollback

สุดท้าย ตรวจสอบยอดเงินใน Account

id	account_number	balance	owner_name
1	1234567890	1500	เจตนิพิฐ แก้วทรัพย์

จากภาพแสดงผลการตรวจสอบข้อมูล Account อีกครั้ง หลังจากทำการฝากเงิน โดยกำหนดให้เกิด error หลังดำเนินการ พบว่ายอดเงินคงเหลือ (balance) เท่ากับ 1500 ซึ่งเปลี่ยนแปลงจากยอดเงินเดิม แสดงว่าระบบไม่มีการ Rollback กรณีที่ดำเนินการไม่สำเร็จ

เปรียบเทียบมี @Transactional กับไม่มี @Transactional

จากการทดลอง เมื่อกำหนด @Transactional ให้กับเมธอด deposit() แม้ว่าระบบจะบันทึก Account และ DepositTransaction สำเร็จแล้ว แต่เมื่อเกิด RuntimeException("Test Rollback") ระบบจะทำการ Rollback การเปลี่ยนแปลงทั้งหมดภายใน Transaction ส่งผลให้ยอดเงินใน Account กลับเป็นค่าเดิมและรายการ DepositTransaction ที่สร้างขึ้นถูกยกเลิก

ในทางกลับกัน เมื่อทดลองลบ @Transactional ออก การบันทึก Account และ DepositTransaction จะไม่ได้อยู่ภายใต้ Transaction เดียวกัน เมื่อเกิด RuntimeException หลังจากบันทึกข้อมูลแล้ว ระบบจึงไม่สามารถ Rollback การเปลี่ยนแปลงทั้งหมดได้ ทำให้ข้อมูลที่บันทึกไปแล้วอาจยังคงอยู่ใน Database แม้ Postman จะแสดง 500 Internal Server Error

7. คำถามท้าย Lab

7.1 @Transactional มีหน้าที่อะไร?

ตอบ @Transactional ใช้กำหนดให้การทำงานหลายขั้นตอนใน Database อยู่ภายใต้ Transaction เดียวกัน โดยระบบจะทำงานทั้งหมดให้สำเร็จพร้อมกัน หรือหากเกิดข้อผิดพลาดก็สามารถ Rollback การเปลี่ยนแปลงทั้งหมดกลับไปเป็นสถานะเดิมได้

7.2 เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ตอบ เพราะการฝากเงินมีการแก้ไขข้อมูลมากกว่าหนึ่งส่วน ซึ่งข้อมูลทั้งสองส่วนควรสอดคล้องกัน หากบันทึกยอดเงินสำเร็จแต่การบันทึกประวัติการฝากเงินล้มเหลว จะทำให้ข้อมูลไม่ตรงกัน ดังนั้นจึงควรใช้ Transaction เพื่อให้ สำเร็จทั้งหมดหรือยกเลิกทั้งหมด

7.3 COMMIT และ ROLLBACK ต่างกันอย่างไร?

ตอบ COMMIT คือ การยืนยันการเปลี่ยนแปลงและบันทึกข้อมูลลง Database อย่างถาวร ส่วน ROLLBACK คือ การยกเลิกการเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction และคืนข้อมูลกลับไปก่อนเริ่ม Transaction

7.4 Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ตอบ ความสัมพันธ์แบบ One-to-Many (1:N) หมายถึง

- Account 1 บัญชี สามารถมี DepositTransaction ได้หลายรายการ
- DepositTransaction 1 รายการ จะเป็นของ Account เพียง 1 บัญชี

7.5 เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ตอบ หาก Error ทำให้ Transaction ถูก Rollback การเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction จะถูก ยกเลิกทั้งหมด ดังนั้นจะไม่เกิดกรณีที่ยอดเงินเพิ่มแล้ว แต่ไม่มีประวัติการฝากเงิน เพราะข้อมูลจะถูกย้อนกลับทั้ง Transaction

7.6 จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ตอบ เมื่อลบ @Transactional ออก การทำงานแต่ละส่วนอาจถูกบันทึกแยกกัน หากขั้นตอนแรกสำเร็จ แต่ขั้นตอนถัดไปเกิด Error ข้อมูลที่บันทึกไปแล้ว อาจไม่ถูก Rollback ทำให้เกิด ข้อมูลไม่สอดคล้องกัน (Data Inconsistency)